

A BEGINNER'S FIELD GUIDE

Getting started with automation

How I would start with n8n today: the core tools and nodes, how to build and grow a workflow that holds up, and how much to lean on an LLM along the way.

By **Felix Sattler**

Verified n8n creator · 50+ production workflows shipped

Who am I

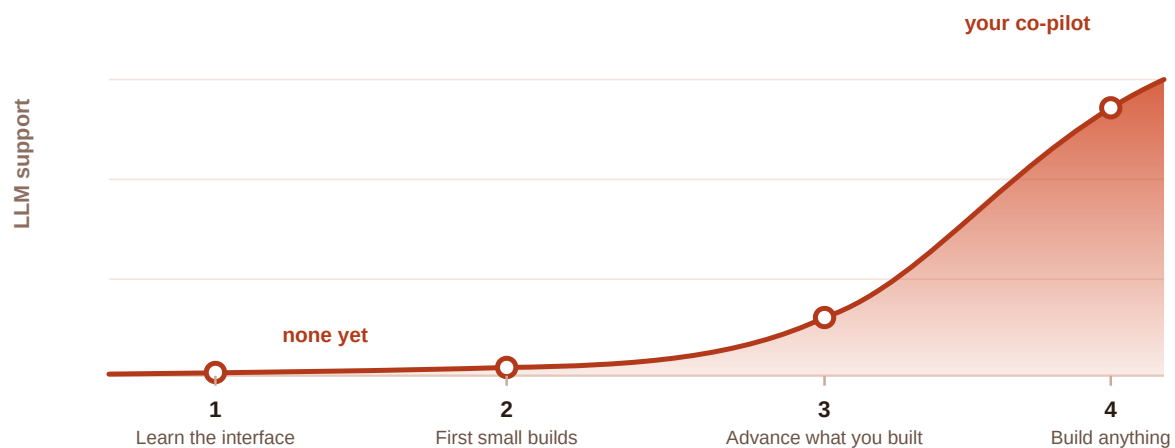


I'm **Felix**, a verified n8n creator. I've shipped 50+ production workflows for real clients, and I publish templates, guides and walkthroughs so other builders can skip the mistakes I made along the way. This guide is not theory: it is the same route I would take again if I were opening n8n for the first time tomorrow.

Find me on n8n.io/creators/easybits · reddit.com/user/easybits_ai

How your journey should look

The four stages to move through, and how much to lean on an LLM at each one.



The orange area is how much you should rely on an LLM. It stays at zero on purpose through your first small builds, then grows naturally as your workflows get genuinely complex.

How I started my journey

1

Learn the interface first

Before building anything real, understand how n8n actually works: how to add nodes, what the basic nodes do, and how data has to flow through a workflow: **trigger**, then **processing**, then an **output** (an interface or an API). Watch a few intro videos until the interface stops feeling foreign.

The n8n beginner course is the best starting point:

youtube.com/watch?v=4BVTkqbn_tY (beginner course playlist)

2

Build something tiny (2 to 4 nodes)

Pick the smallest useful thing and build it end to end. My very first workflow was a drinking reminder: a Schedule trigger firing a Slack notification. Two nodes. That is all you need to feel the trigger to action loop click into place. Do this phase entirely on your own, without an LLM.

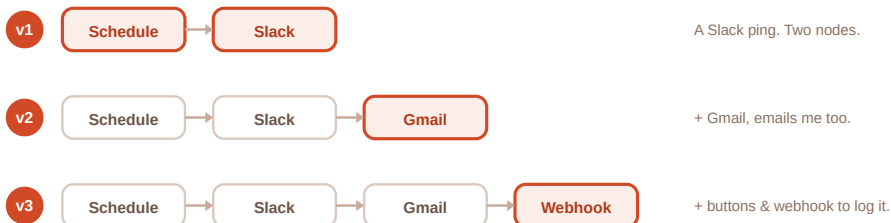
3

Advance what you already built

Now grow it. Look at a workflow you have running and ask what could make it better. This is where community matters, and other builders will hand you angles you would never think of.

Start small. For my drinking reminder I first added a Gmail node so updates also hit my inbox, then Slack buttons to log whether I drank the water, which forced me to learn webhooks (see below).

How my drinking reminder grew, v1 → v3



Same workflow, three versions, each step just added one node (highlighted).

Early on, resist leaning on Claude or any LLM: you want to understand what is happening and why. Once the workflow gets genuinely complex, that is the moment to start using Claude for questions and learning.

4

Build anything

From here you can approach almost any project. Keep one rule: the first round of thinking is always yours. Plan the workflow yourself, then execute it with Claude's help.

Practical tip: if you get stuck, export the workflow JSON and hand it to Claude. It can then see the whole picture at once, and that context often unblocks things fast.

A good basic stack

These few tools and their nodes cover the vast majority of what I build. Learn them and you can handle most real use cases.

n8n The automation engine everything else plugs into. This is home base.

Google API

Sheets, Docs, Drive and Gmail: your everyday data stores and surfaces. Read and write rows, generate documents, move files, send mail.

Slack

A clean interface for anything work-environment related: notifications, approvals and alerts that land where your team already is.

Telegram

A great starting interface. Huge user base, and setup is genuinely simple thanks to BotFather.

easybits Extractor

For document processing use cases: invoice extraction, CV extraction and similar. It pulls structured data out of messy PDFs and images.

Core nodes to learn first

A handful of nodes show up in almost everything I build. Get comfortable with these early.



Form Trigger

It bakes straight into the workflow JSON with no extra setup, unlike Gmail or Slack. Anyone can import your workflow and use the exact same form without configuring a thing.



Respond to Webhook

This one feels like magic. You can design the actual page (HTML) that shows up when the workflow responds, and suddenly your workflow has a real front end.



IF

Perfect for the simple checks you will reach for constantly: duplication checks, document-type checks, and so on.



Switch

Your router. Send data down different paths based on a class or the output of a previous node.

A few more core nodes

These round out the set. They show up constantly once your workflows start doing real work.



Edit Fields

Where you shape data: rename fields, set defaults, derive new values (this is the node formerly called Set). Gotcha: never put a leading = in front of a `{{ }}` in the value field, or your output quietly turns into `[object Object]`.



Google Sheets

The lightweight database under most of my builds, the storage layer you do not have to set up. Gotcha: everything comes back as a string, so a number or date read from a sheet needs converting before you calculate or compare on it. When you outgrow a sheet (real volume, or many writes at once), Supabase is the natural next step up.



Loop Over Items

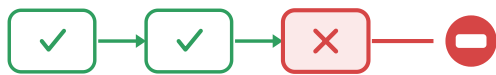
Essential for batch processing, and honestly the node that best visualises what a loop is actually doing while it runs.

Error handling and logging

Once your workflows do real work, the question stops being "does it run?" and becomes "what happens when it doesn't?" This is the part most beginners skip and later regret.

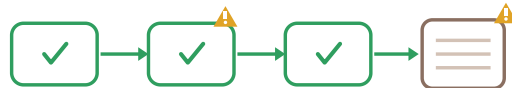
There are two kinds of failure. Loud ones throw a red error and stop the run. Quiet ones are more dangerous: the run finishes green, but empty or mis-parsed data flows on unnoticed.

● Loud failure



Stops the run. Easy to spot.

● Quiet failure



Finishes green, bad data flows on.

First, learn to read the execution log. Every run is saved as an *execution*. Open the Executions tab, click a run, and step through each node to see exactly what data went in and came out. When something breaks, start here: find the failing node, then check what the node *before* it passed on. Most bugs are visible in seconds, no guessing.

The screenshot shows the workflow interface for 'Extract CV into Google Sheets (easybits)'. The 'Executions' tab is selected, showing a list of runs. The top run is 'Aug 24, 11:04:15' with a red 'Error in 3.13s' status. Below it are two successful runs. The workflow diagram at the bottom shows three nodes: 'CV Upload', 'easybits Extract CV', and 'Fan-out CV Data'. A red arrow points to the 'Executions' tab with the text 'Every run is logged here'.

Every workflow has its own **Executions** tab (top right). Open it to see each run (green succeeded, red failed) and click one to step through it node by node.

Building resilient workflows

Most of error handling comes down to one job: turning silent failures loud. These three habits do most of the work.

Catch loud failures at the node. Every node has an "On Error" setting in its Settings tab. Switch it from "Stop the Workflow" to "Continue (using error output)" and the node grows a second, red output pin. Wire that pin to a fallback: a retry, a backup API, or a Slack alert. The classic trap: turn on the error output but leave that red pin unconnected, and the flow dies silently. Always connect it.

Catch quiet failures yourself. For bad-but-not-crashing data, add an IF node right after the risky step and test your critical field with "is empty". Good data continues; empty data branches to an error path. With the easybits Extractor this is especially clean: it returns an empty value when it can't read a field, so "is empty" is a reliable failure signal.

Log every failure, and make it loud. Whatever lands in the error path, do two things. First, ping yourself on Slack with the workflow name and what broke: do not just log it quietly, an alert you actually see is what lets you fix the problem before the client notices, which matters most in a build's first few weeks. Second, append a row to a "failures" Google Sheet with the input and the reason it failed. After a week the log shows which errors really repeat, and the fix is usually one bad assumption rather than a hundred random errors, so you fix root causes instead of firefighting.



n8n's built-in safety net

Build one workflow starting with an Error Trigger node, then point your others at it under Options, then Settings, then Error workflow. When any fails, this handler runs automatically with the failure details, ready for a Slack alert or log. One handler covers every workflow you build.

docs.n8n.io/build/flow-logic/handle-errors-gracefully

Handy trick: the Stop And Error node. Sometimes a run technically succeeds but the data is wrong (a quiet failure you caught with an IF check). Drop in a Stop And Error node on that branch and it deliberately throws an error, which stops the run and hands it to your global error workflow, so a caught quiet failure gets treated exactly like a loud one.

How I test before I ship

One clean run is not a test, it is a demo. Before a workflow goes anywhere near real client data, I put it through the same short routine every time.

1 Build the test set from the ugliest real samples

A tidy PDF tells you nothing. I deliberately collect the messy stuff: blurry phone snaps, skewed scans, stamped copies, documents in two languages. Get it working on the worst inputs and the clean ones look after themselves.

2 Generate the variety you do not have

With only one or two real layouts on hand, I have an LLM redraw the same data in different arrangements, and also produce copies with fields left blank on purpose. That way I see how the flow copes with formats and gaps I have not met yet, rather than finding out when a client sends one (examples on the next page).

3 Break it on purpose

Then I feed it the wrong things on purpose: an empty upload, the wrong kind of document, a field that simply is not there. I want to confirm those get routed to review or fail loudly, instead of slipping an invented value quietly downstream.

4 Run a batch, not a single document

A single success proves little, so I run twenty or thirty through at once and watch for drift. The odd edge case almost always surfaces in a batch, and I would much rather it surface on my screen than on the client's.

5 Check the output against the source, not just that it ran

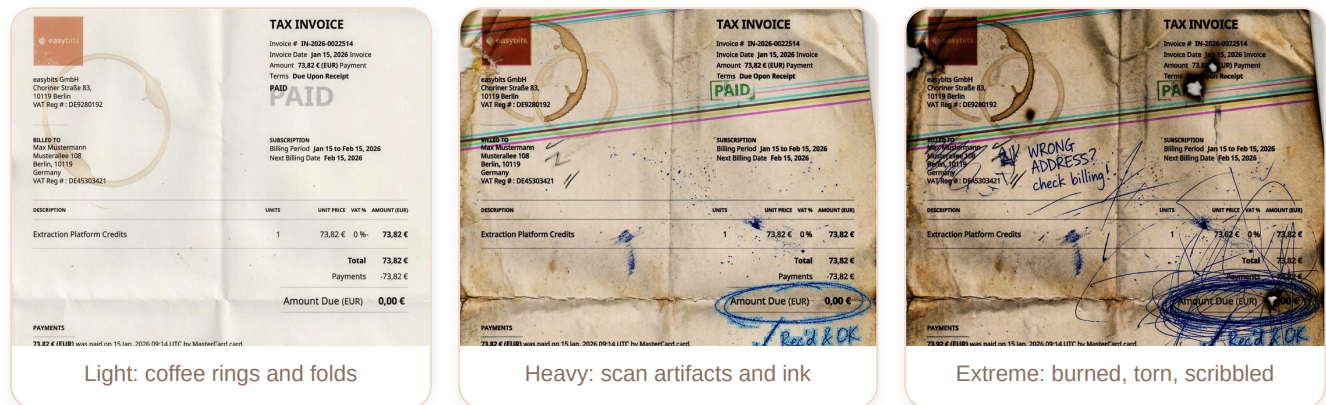
Green only means it finished, not that it is correct. I put a few results side by side with the original and check the fields that carry weight: totals, IDs, dates. Those are the ones that cost real money when they slip through wrong.

For big flows: on larger builds I walk Claude through the flow, share a flow chart, and ask for a testing overview as a PDF. It surfaces edge cases I would never have listed myself, and leaves me a clean checklist to work through one by one.

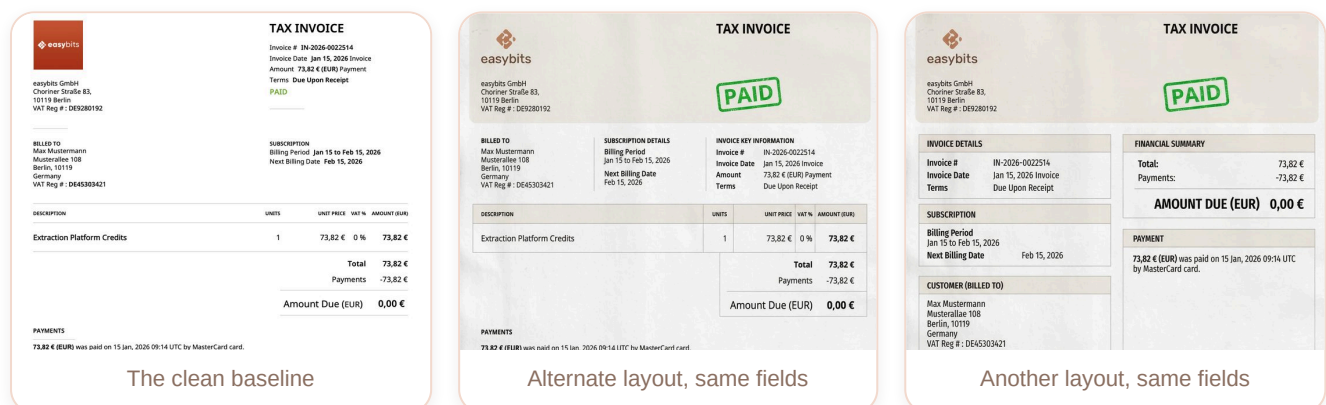
What a real test set looks like

The example set behind steps 1 and 2: one clean invoice, degraded on purpose, plus the same data points redrawn in different layouts. If extraction holds up against these, the clean documents are the easy part.

The same invoice, stressed on purpose



Same data points, different layouts



Tip: I generated this whole set from a single clean invoice. I go into how I build these test documents in my stress test video: youtu.be/bOODILPEdho

Agentic vs deterministic

Agents are everywhere right now, and they have earned their place. But they are not the answer to everything, and reaching for one by default is an easy, expensive habit to pick up.



Deterministic

You define every rule

- Predictable and testable
- Cheap and fast
- Never hallucinates
- Only as smart as your rules



Agentic

An LLM decides at runtime

- Handles ambiguity & open-ended tasks
- Can hallucinate
- Token-heavy, so pricier
- Less predictable

MY RULE OF THUMB

Can you sketch the entire process on paper: every decision, every rule?

YES →

Build it **deterministically**. Predictable, cheap, and easy to debug.

STILL BLIND SPOTS →

Decisions you cannot pin down cleanly? That is where an **agent** really shines.

The cost of reaching for an agent

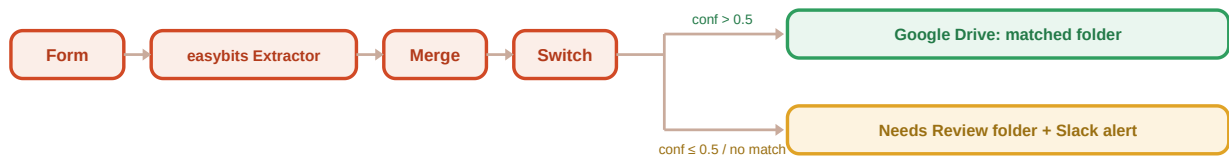
Watch the token burn. An agent does not run a fixed set of steps: it calls the model to decide what to do next, often several times per item, and every one of those calls is tokens you pay for. A deterministic flow calls a model zero times, or exactly once where you truly need it, so its cost stays flat and predictable. Run one document through an agent and the bill looks like nothing. Run a thousand a month, with the agent re-reading context and retrying, and it becomes a real line item, for work a Switch node would have done for free. Used well, agents are worth every token. Reached for by default, they quietly turn a cheap job into an expensive one.

A build I kept deterministic

The clearest way to see the trade-off is a real one. Here is a workflow where the agentic version genuinely looked tempting, and why I did not ship it.

The setup. One of our published workflows sorts uploaded invoices into the right Google Drive folder. A form takes the file, the easybits Extractor returns a document type and a confidence score, a Merge keeps the original file attached, and a Switch routes each type to its folder. Anything under the confidence threshold goes to a Needs Review folder with a Slack alert, so nothing is ever filed on a guess.

Invoice classifier, kept deterministic



Types the Switch routes: Medical, Restaurant, Hotel, Trades, Telecom, each to its own folder.

Fixed categories, fixed folders. The Switch decides, not a model, so an invoice never lands somewhere I did not define.

Then I tried the agentic version. Same task, but an AI Agent with Google Drive tools deciding where each file should go. It worked, yet it wandered: documents landed in the wrong folders, and once the agent even created a brand new folder that was never meant to exist. I also hit a hard limit (tool nodes under an agent cannot reach the binary from the main branch), so the file handling had to be reworked. More tokens, more moving parts, for a job with fixed categories.

It comes down to how each one fails. Deterministic flows fail loud, while an agent can fail quietly, still reporting success while it files things in the wrong place. With fixed categories, the Switch keeps every failure visible, and that is worth more than the flexibility I would have gained.

Things to avoid

Overusing Claude. Full MCP integration, or asking Claude to return an entire finished workflow, can work, but you will not learn anything, and you end up completely reliant on the LLM for every build. The skill you are trying to develop never gets a chance to form. The simplest test of independence: when an error hits, can you work it out without Claude?

Starting with a project that is too big. Your first real build should not be the ambitious end-to-end system. Bite off something too complex and you stall, then you hand the whole thing to Claude to untangle, and you are right back to leaning on the LLM for every step instead of learning. Start small, get it running, then grow it. The scope is what keeps you in control.

Importing templates without reading them. The n8n template library is a fantastic resource and a real head start. But do not wire a template into your data and switch it on untouched. Open it, walk through it node by node, and follow how the data actually flows. That is how you learn the pattern, confirm it does what you think, and make sure it is safe before it ever runs on real data.

The test

One last gut check before you go build, the one I still run on myself.



Claude · ChatGPT · Gemini · ...

The test: are you already too reliant on your LLM?

Think about the last error you fixed in a workflow. How did you approach it?

Here is how I do it. I open the execution logs first. What did the node *before* the failing one output? Was the structure right? Are there datapoints missing that the failing node needs in order to run?

If your honest answer is "**I just asked Claude**", that is your signal. Try the manual path first next time.

Where to go from here

The whole path in one line: learn the interface, build something tiny, grow it, then build anything, leaning on an LLM only once your workflows have earned it.

1

Learn the interface

2

Build something tiny

3

Advance what you
built

4

Build anything

See it all in practice



I keep 25+ free workflows on GitHub, from small workflows to full production builds. Clone them, break them, learn from them. A star on the repo helps me out a lot if any of them save you time.

github.com/felix-sattler-easybits/n8n-workflows

Watch the next steps



Multiple explainer videos on our YouTube channel walk you through the next steps, from your first build to more advanced workflows.

youtube.com/@easybits_ai

Questions as you go?

People reach out to me about this all the time, and I'm happy to help. The easiest way is to grab a free 15-minute call.

[Book a free 15-minute call](#)

Or email me directly at felix.sattler@easybits.tech

One last thing. The road into n8n and automation is not always an easy one, and getting active in the communities, which is a real part of the journey, means seeing incredible builds and a lot of strong opinions argued out in the open. Whenever you hit one of those posts that makes you wonder if you are already too late, remember where those people are standing: the n8n and automation communities sit right at the front of the AI and automation wave, and most of the working world is nowhere near it yet. Surveys still put the share of workers stuck on repetitive, automatable tasks at around 94%, so the vast majority have not touched any of this.

Everyone started somewhere, and a busy feed does not mean the door has closed. There is a staggering amount you can build and automate with n8n once you begin. Go at your own pace, keep building small, and before you notice it you will be the one weighing in on where automation goes next.

Best,

Felix